

AME556 Robot Dynamics and Control

Final Project Report

Group 5

Hojung Lim: 1661958467

Seong Wook Kim: 1268805670

Link to Simulation Videos & Codes

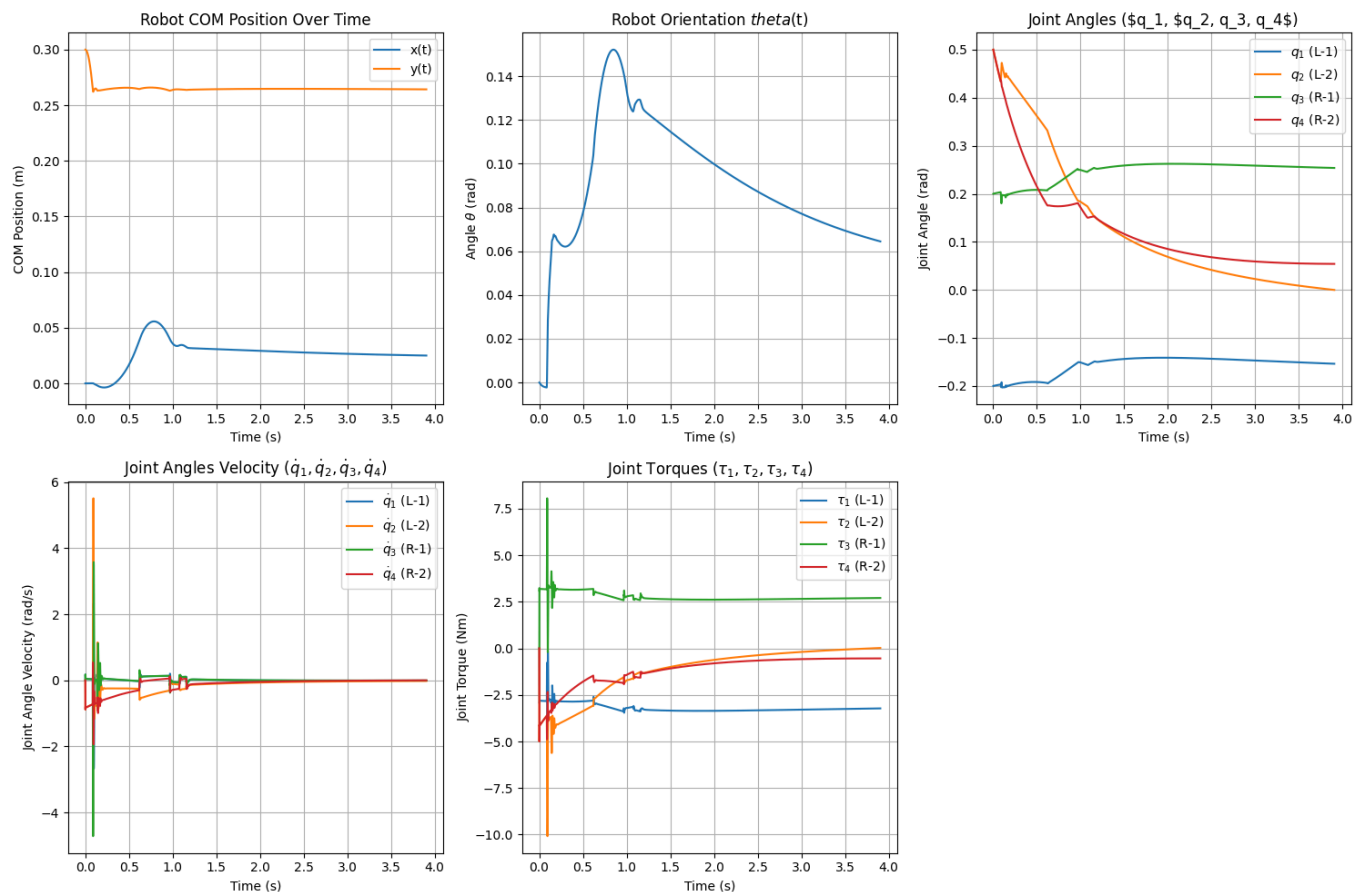
https://drive.google.com/drive/folders/1os5ldJegevlPCxhxn09U_RNOMumtupPp?usp=drive_link

Task1

Using Python, we created two helper functions to check the constraints and return the violation flag when the constraints were violated.

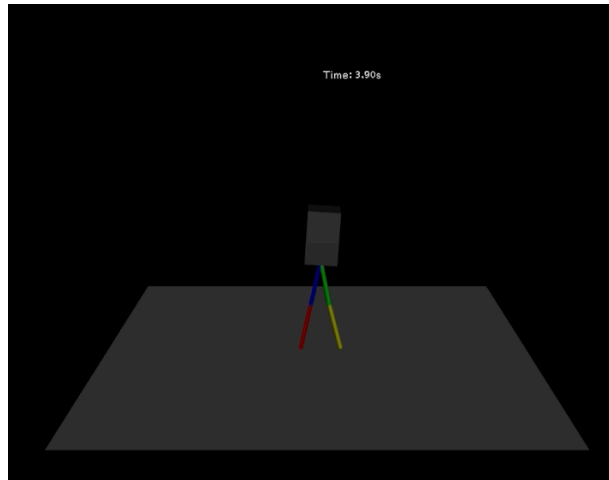
- `check_constraints`: check if the hip joints, knee joints, joint velocity, and torques violate the constraint or not. Returns a dictionary of boolean values.
- `Get_violation`: returns a violated element(s) when the program terminates.

Including these two functions, Controller class was created to get the id of each component of the bipedal robot, like x and y positions, and q values for each link. Also, the `stand_controller` function calculates the torque for each link using PD control. The K_p and K_d values were selected after testing.



Plot for Task1

As shown in the joint angles plot, joint angle q_2 becomes smaller than zero, and the simulation immediately terminates. The exact q_2 value at that time was $-1.957238909344495e^{-5}$, which is about -0.0011214152900422792 degrees. The violated constraint occurred at $t = 3.9095$ seconds. Furthermore, the joint torques vary from $-10(\text{N}\cdot\text{m})$ to $7.5(\text{N}\cdot\text{m})$ which do not violate the constraints.

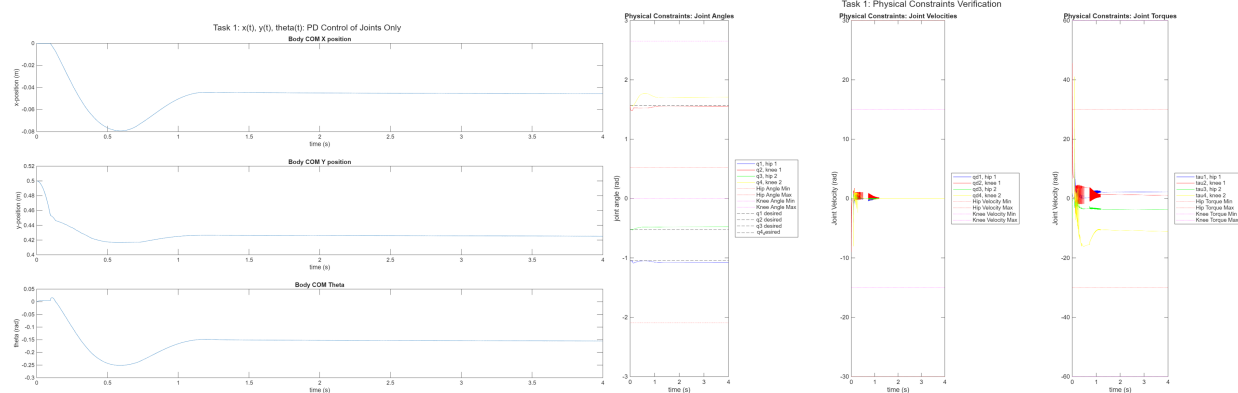


Simulation time when program terminates

To display simulation time on the Mujoco viewer, a helper function “add_text” was created.

Matlab Approach:

We were able to approach Task 1 using both Matlab and Python as well. This can be done by creating a function (“get_state”) in the python code that assigns the state data from Mujoco (MjData) which can later be called in Matlab. In both Task 1 and Task 2, the constraint violation call out was created through a series of if statements. When the state data, $q = [x; y; \theta; q_1; q_2; q_3; q_4]$ & $\dot{q} = [\dot{x}; \dot{y}; \dot{\theta}; \dot{q}_1; \dot{q}_2; \dot{q}_3; \dot{q}_4]$ & $\tau = [\tau_1; \tau_2; \tau_3; \tau_4]$, is read in Matlab from Python, the states are compared with the physical constraints of the joints. If any of the state data violates the physical constraints, the code will terminate the simulation and call out the specific constraint violation. When all of the conditions are met, we can feed the state data, q & $\dot{q}$, to our PD controller u_{pd} , where $u_{pd} = K_p \text{_{joint}} * (q_{desired} - q) + K_d \text{_{joint}} * (\dot{q}_{desired} - \dot{q})$ & $u = -[K_p \text{_{joint}}, K_d \text{_{joint}}][e; \dot{e}]$. In Task 1, the desired joint angles were set to be the same as the initial conditions. The two plots below show the body COM (x, y, θ) and joint constraints (joint angle, velocity, torque) over time when the initial conditions and desired joint angles are equal to $q_0 = q_{desired} = [-\pi/3, \pi/2, -\pi/6, \pi/2]$. $K_p \text{_{joint}}$ and $K_d \text{_{joint}}$ were determined by trial and until successful simulation was found, i.e. until the robot did not fall. Initial conditions were also found by this method.



Plot on the left shows Body COM position over time. Plot on the Right shows joint angle, velocity, & torque over time

Note: Matlab and Mujoco animations of physical constraint violation at different initial conditions can be found in the folder link above. All codes and figures can be found in the folder link above.

Note: 2D_Biped_Robot.xml for Task 1 and Task 2 are different. Task 1 model enables collision between the legs. Task 2 model enables no collision between the legs.

Task2

Standing Up and Down

The physical variant of this task is to stand at the desired state of $y_d = 0.45\text{m}$, $\theta_d = 0\text{ rad}$ for 1 second. After that, the robot should move the body up and down, the desired state $y_d = 0.45\text{m}$ to 0.55m in 0.5 second , and moving from $y_d = 0.55\text{m}$ to 0.4m in 1 second .

Matlab Approach:

Task 2 Standing Up and Down can be primarily broken down into three main parts:

1. Following a Trajectory: $y_d \rightarrow 0.55\text{m}$ in 0.5s , $y_d \rightarrow 0.55\text{m}$ in 1s
2. Calculating the desired joint angles by Inverse Kinematics (Difficult)
3. QP Control for Robot Balance (Joint angle, velocity, torque limits)

In Task 2, the body COM is required to follow along a desired trajectory of y_d and $\theta_d = 0$ along a given time frame. The trajectory can be broken down into five phases:

- 1.1) Transition period of 2s from initial condition to starting position, $y_d = 0.45$. Initial joint conditions were not selected to be at $y = 0.45\text{m}$ at $t = 0$.
- 1.2) When the transition period is over and the robot has stabilized from the initial condition, hold for another 0.5s . This step is to ensure stability before rising up.
- 1.3) When 0.5s has elapsed. set the rise in trajectory from 0.45m to 0.55m in 0.5s to be a cubic polynomial for smoother joint movements than compared to that of a pure Linear

rise and drop trajectory which would cause sharp movement in the joints, possibly causing joint constraint violations.

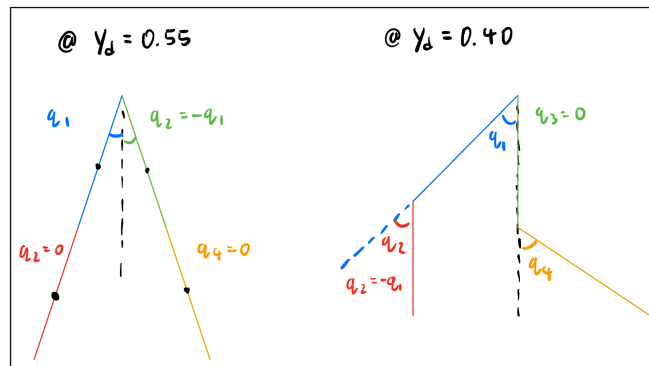
1.4) When the desired height of 0.55m has been met, hold for 0.5s. Similarly to 1.2), this step is to ensure stability before dropping down.

1.5) Once the stabilization time has elapsed, set the drop in trajectory from 0.55m to 0.40m in 1s to be cubic polynomial for smoother joint movement on the way down. We will then hold for 0.5s to ensure the robot has remained stable.

Obtaining the desired torque by Inverse Kinematics proved to be the most difficult given the many physical joint constraints. Ideally, a symmetric robot shape robot such as a pyramid shape where joint knee angles q_2 & q_4 are approximately equal to zero. However this shape and other symmetric designs proved to be difficult to implement because of the limited hip joint angle, $q_{hip} \leq 30^\circ$. This predicament led us to a lunge-like figure used in Task 2 as shown in the picture below. The following constraints were used to solve for the desired joint angles ($q_{desired}$) by Inverse Kinematics:

- @ $y_{desired} = 0.55m$, $l_{leg} = .425 \approx .42$ (Prevent joint singularities)
 - $q_1 = -\arccos(l_{leg}/(2*Li))$
 - $q_3 = -q_1$
 - $q_2 = q_4 \approx 0$
- @ $y_{desired} = 0.44m \rightarrow l_{leg} = .275m$
 - $q_2 = -q_1$
 - $q_3 = 0$
 - $q_4 = \arccos(l_{leg}/L-1)$

These start and final constraints were used to interpolate the relationship between the hip & knee joints and $y_{desired}$ to maintain lunge-like pose.



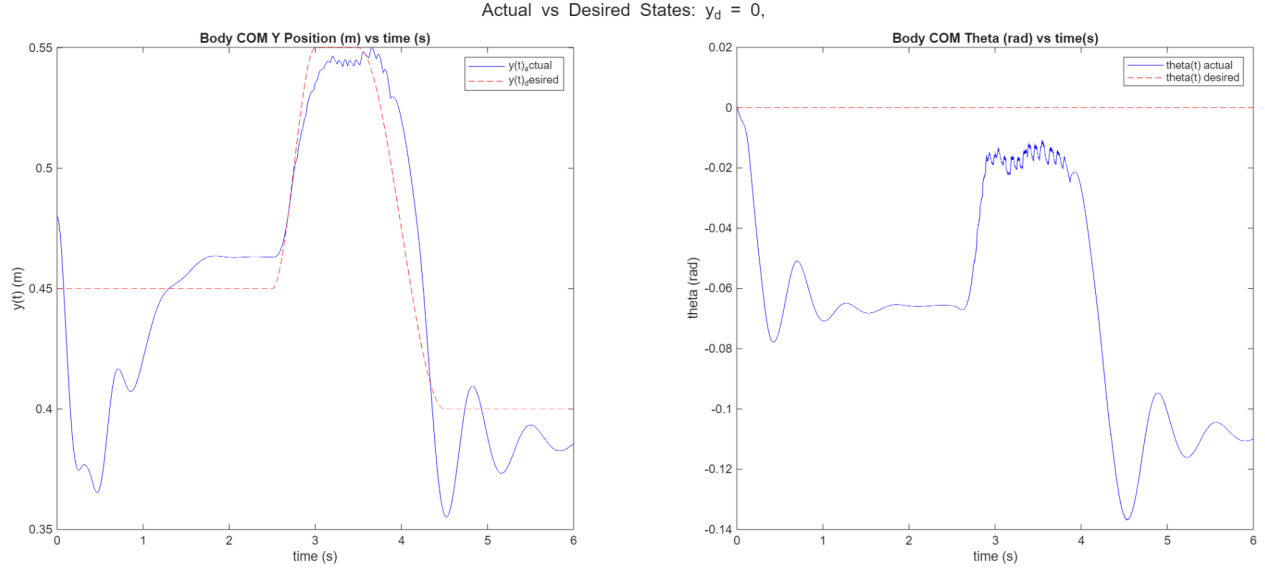
Picture showing the ideal final positions of robot (ideal range of joint movement)

QP Control was set using hip and knee joint angle, velocity, and torque constraints. Using approximate mass matrix, $M = M_{diag}$, and the approximate equation of $D\ddot{q} + C\dot{q} + G = \tau$ where $C\dot{q} \approx 0 \rightarrow \ddot{q} = \tau/M - G/M$:

- $A_{qp} = [I(4 \times 4); -I(4 \times 4); 1/M(4 \times 4); -1/M(4 \times 4); \pm 1/M((n = 1:8) \times 4)]$

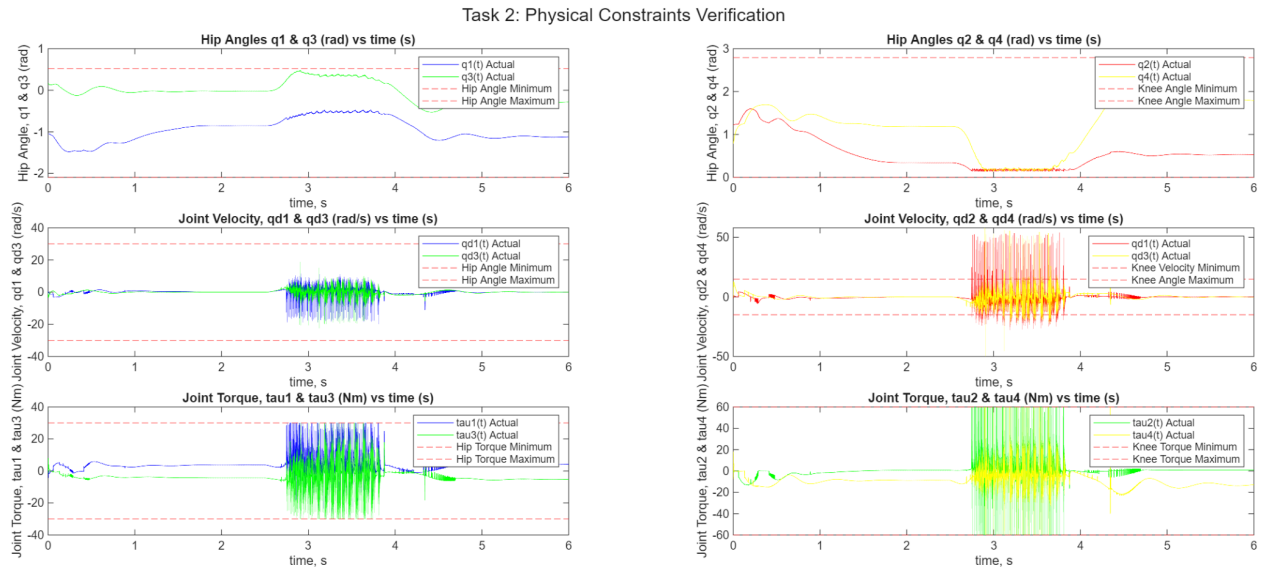
- $\mathbf{bqp} = [\tau_{\text{Max}} (4 \times 1), \tau_{\text{min}} (4 \times 1); \dot{q}_{\text{max}} (4 \times 1); \dot{q}_{\text{min}} (4 \times 1); \pm \tau_q (n = 1:8) \times 4)]$
 - where, $\pm \tau_q (n = 1:8) \times 4 \rightarrow \pm \tau_q = \pm \ddot{q} \pm G/M$
 - Note: $\mathbf{Aqp}$ has variable row sizes between $n = 16:24$. τ_q is only activated when joints are near joint limits.
- $\mathbf{H} = \mathbf{I} (4 \times 4)$, $\mathbf{f} = \tau_{\text{desired}}$
- Input saturation, $\tau_{\text{lower_bound}}$, $\tau_{\text{upper_bound}}$
- $\mathbf{U} = \text{quadprog}(\mathbf{H}, \mathbf{f}, \mathbf{Aqp}, \mathbf{bqp}, [], [], \tau_{\text{lower_bound}}, \tau_{\text{upper_bound}})$

The plot below shows the plot of Body COM $y(t)$ and $\theta(t)$ relative to its intended trajectory.



The Plot on the left shows the Body COM $y(t)$ compared to its desired trajectory. The Plot on the right shows the Body COM $\theta(t)$ compared to its desired trajectory, $\theta_d = 0$

The figure below shows the joint angle, velocity, and torque over time against its tolerance.



Plot showing joint angles, velocity, and torque over time

Kp_joint and Kd_joint were determined by trial and until successful simulation was found, i.e. until the robot did not fall. Initial conditions were also found by this method.

Note: Matlab and Mujoco animations of physical constraint violation at different initial conditions can be found in the folder link above. All codes and figures can be found in the folder link above.

Walking Forward and Backward

Using Python, two helper functions "cal_desired_q" and "height_traj" were created to calculate the stable joint angles, and to check the desired y position trajectory of the robot's COM over time. The helper function "cal_desired_q" uses inverse kinematics to calculate the joint angles.

```
def cal_desired_q(self, z_d):
    """
    calculates inverse kinematics for 2-link leg
    x1 = trunk_x + l_leg*sin(q1)
    y1 = trunk_y - l_leg*cos(q1) - 0.125
    x2 = x1 + l_leg*sin(q1 + q2)
    y2 = y1 - l_leg*cos(q1 + q2)

    at t=0 x2 = y2 = 0
    ...
    # foot position when stable
    foot_x_stable = 0.25

    foot_x_right = foot_x_stable
    foot_x_left = foot_x_stable

    h = z_d - 0.125
    l_tmp_right = np.sqrt(foot_x_right**2 + h**2)
    l_tmp_left = np.sqrt(foot_x_left**2 + h**2)

    # prevents math errors
    l_max = 2 * self.l_leg * 0.999
    if l_tmp_right > l_max: l_tmp_right = l_max
    if l_tmp_left > l_max: l_tmp_left = l_max

    alpha_1 = np.arctan2(foot_x_left, h)
    cos_gamma_1 = (self.l_leg**2 + l_tmp_left**2 - self.l_leg**2) / (2 * self.l_leg * l_tmp_left)
    gamma_1 = np.arccos(np.clip(cos_gamma_1, -1, 1))
    q1_d = gamma_1 - alpha_1

    alpha_3 = np.arctan2(foot_x_right, h)
    cos_gamma_3 = (self.l_leg**2 + l_tmp_right**2 - self.l_leg**2) / (2 * self.l_leg * l_tmp_right)
    gamma_3 = np.arccos(np.clip(cos_gamma_3, -1, 1))
    q3_d = alpha_3 + gamma_3

    # solve for q2 and q4
    beta_2 = np.pi - 2 * gamma_1
    q2_d = -(np.pi - beta_2)

    beta_4 = np.pi - 2 * gamma_3
    q4_d = -(np.pi - beta_4)

    return [q1_d, q2_d, q3_d, q4_d]
```

cal_desired_q function in Python

The total length of the two links when knee joint angle is zero is 0.44m. According to trigonometry, in order to minimize the hip joint angle, the length of the foot stance should be minimized (see figure).

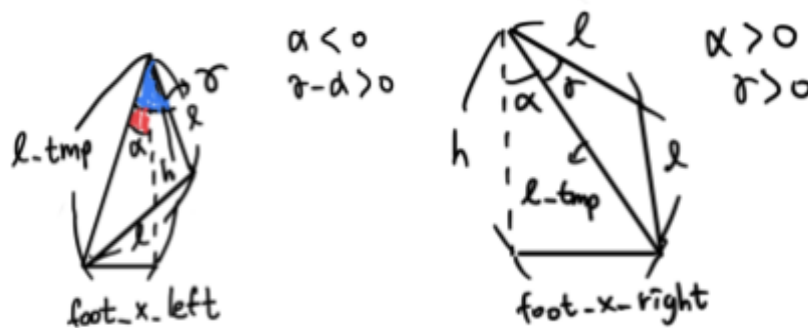


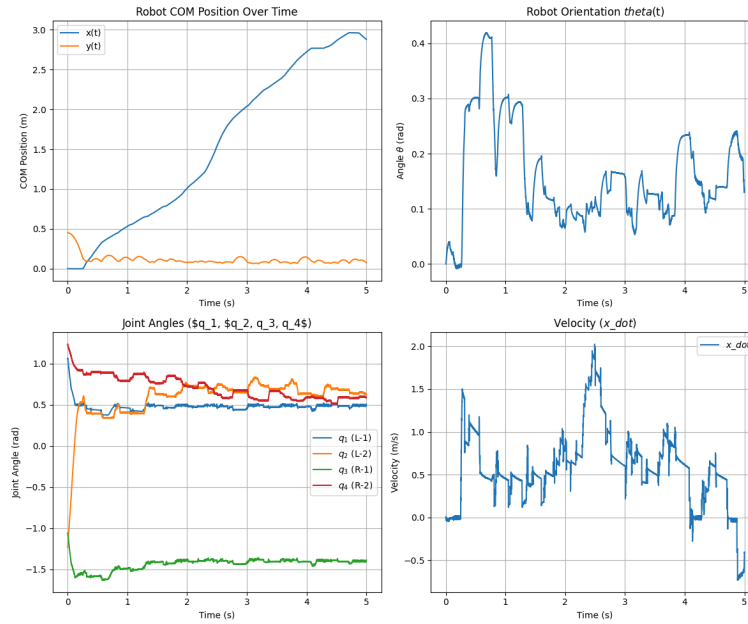
Diagram of Inverse Kinematics 1

Diagram of Inverse Kinematics 2

In order to calculate the alpha and gamma values, trigonometry should be used. The alpha can be calculated using arctan, and gamma can be obtained by using Cosine law. As a result, the joint angle q1 equals gamma - alpha, and the joint angle q3 equals alpha + gamma. If q1 and q3 values are obtained, the joint angles q2 and q4 can be easily calculated.

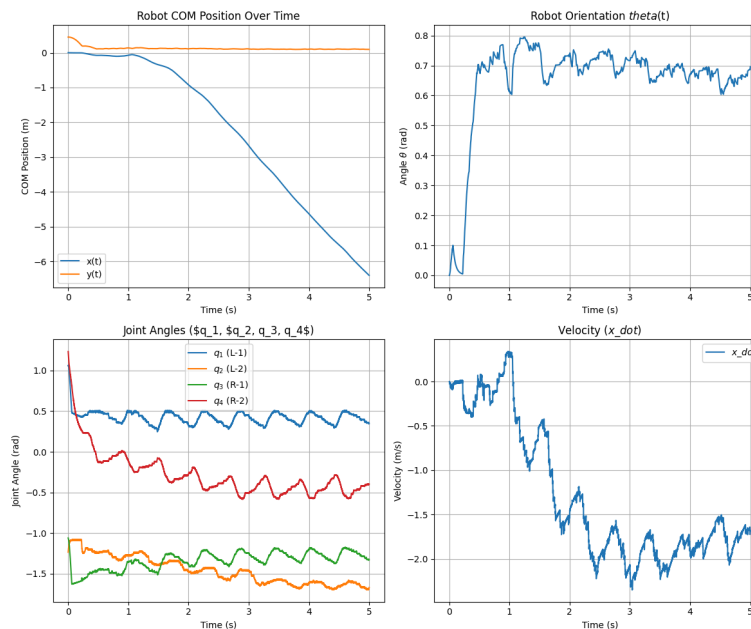
Including these functions, Controller class from task1 was used again, with addition of two controllers, "standing_controller" and "walking_controller". The "standing_controller" takes three parameters, t, z_d, Θ_d , where t is the current time, z_d is the desired y_position, and Θ_d is the desired orientation of the robot body. The y_position is denoted as z since the mujoco 3D

coordinate system, z is the vertical height. Both controllers use PD control to calculate the torques of each link. Also, in the “walking_controller” it takes a boolean parameter, “isForward” so that the robot can walk in an opposite direction when walking backwards.



Plot for task2 walking forward

For a 5 seconds of simulation duration, the forward velocity of the robot reaches up to 2m/s at about $t = 2.5$ seconds. The body orientation, θ , varies from 0 to 0.42 radians, which is about 0 degree to 24 degrees. For 5 seconds, the robot moves about 3 meters in $+x$ direction.

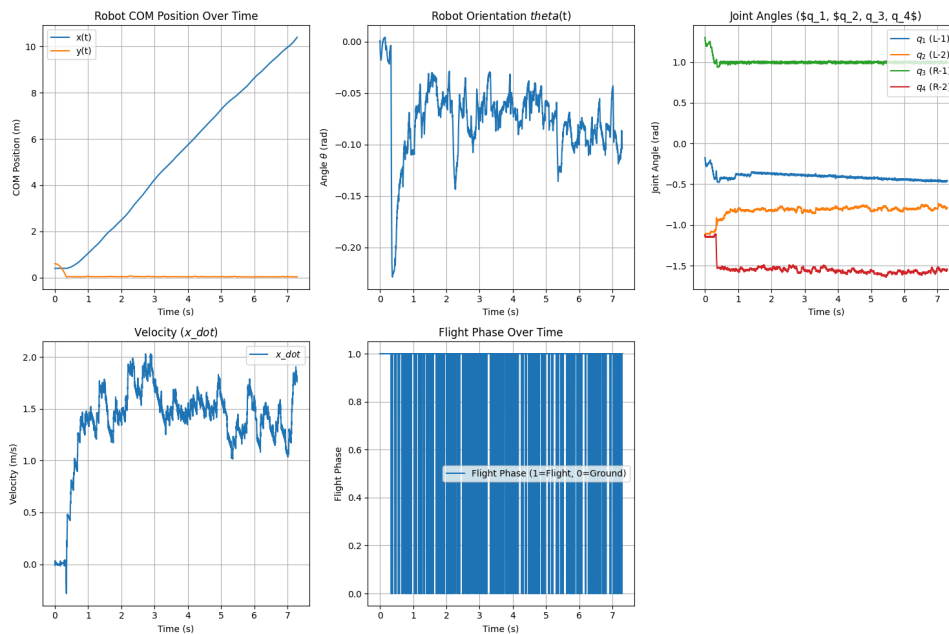


Plot for task2 walking backward

For a 5 seconds of simulation duration, the backward velocity of the robot reaches up to 2.5m/s at about $t = 3$ seconds. The body orientation, θ , varies from 0 to 0.8 radians, which is about 0 degree to 45.8 degrees. For 5 seconds, the robot moves about 6.5 meters in -x direction.

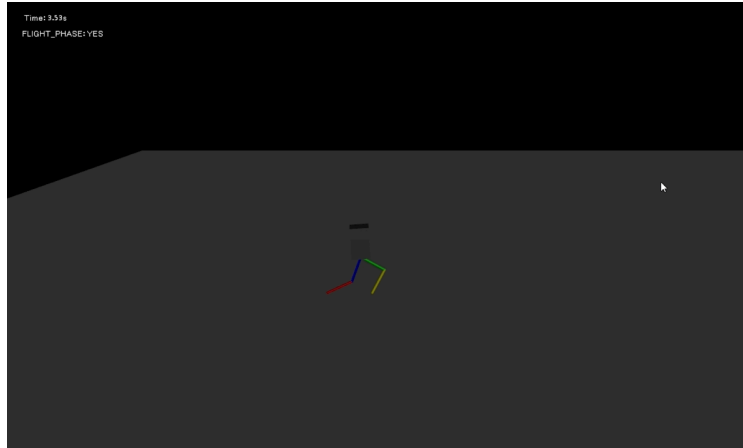
Task3

Using Python, two helper functions "cal_desired_q" and "check_flight_phase" were created to calculate the stable joint angles, and to check the robot's flight phase. The inverse kinematics used in the "cal_desired_q" function is explained in task2 above. Including these functions, Controller class from task1 was used again with addition of more ids. Also, the "running_controller" function calculates the torque for each link using PD control. Considering frequencies and phase for each link, this function calculates the swing of the legs so that the robot can keep its stance to stabilize and move forward at the same time. The K_p and K_d values were selected after testing.



Plots for Task3

As shown in the "Flight Phase Over Time" plot, the value is 1 when the robot is at flight phase, and 0 when both feet are on the ground. The total time taken for 10 meters running was 7.287 seconds. This makes the score of 27.446



Simulation of running task with flight phase and time

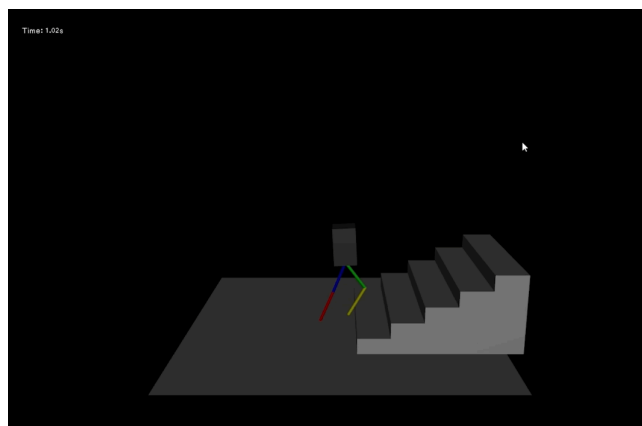
Task4

Using Python, two helper functions “check_foot_contact”, and “check_progress” were created.

- check_foot_contact: This is the same helper function “check_flight_phase” from task3. Only the name has changed.
- check_progress: This function checks if the robot has started climbing stairs. In our model, the world not only contains stairs, but also ground. Thus, the robot’s initial position is not directly on the ground. It starts from a flat ground, and walks to the stairs to climb them.

Including these functions, Controller class from task1 was used again with addition of stair parameters and other helper variables. Also, the “stair_walking_controller” function calculates the torques for each joint when climbing the stairs, and this has a similar structure with the “running_controller” from task3.

However, we did not successfully complete the task. The robot can walk, but its knee joint angles would not lift as high as the stairs, causing collision.



Simulation of stair climbing task with time

Task5

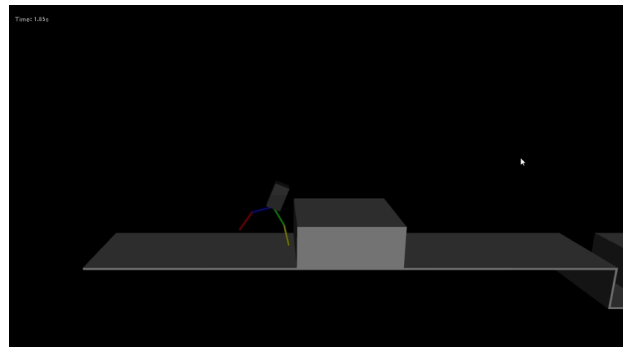
Using Python, The Controller class has a similar structure with the Controller class in task4, with addition of the “should_jump” function and two controller functions, “running_controller” and “jumping_controller”. These two controllers calculate torque for each joint when the robot needs to jump an obstacle or run the flat ground.

The “should_jump” function lets the controller know when to jump, so that the robot can avoid colliding with the obstacles. The obstacles can be distinguished by their x positions, when the robot's x position reaches right before the obstacles, the controller would know that it should jump at that moment.

The “jumping_controller” uses PD control to calculate the torques for each joint when jumping to avoid collision with the obstacles. It has four different jumping states, “crouch”, “thrust”, “flight”, and “land”.

- crouch: At this state, the robot sits down to prepare to jump.
- thrust: At this state, the robot jumps to reach the highest point.
- flight: At this state, the robot is at flight phase, no contact with the ground.
- land: At this state, the robot reaches ground level, the robot should be stabilizing so that it does not fall down.

However, we did not successfully complete the task. Similar to the failure in task 4, the robot could not jump as much as the first obstacle's height. Thus, the robot collided with the obstacle and fell down.



Simulation of obstacle course task with time